

CSS Grid

Two-Dimensional Layouts With Total Control

■■ CSS GRID

■ LEARNING OUTCOME

By the end of this module you will build complex two-dimensional page layouts with CSS Grid — magazine layouts, dashboards, portfolio grids, and full page structures — without any hacks or frameworks.

■ Skills You Will Gain

- Define grid columns and rows with `grid-template-columns/rows`
- Use the `fr` unit and `repeat()` function
- Place items with `grid-column` and `grid-row`
- Name areas with `grid-template-areas` for visual layout code
- Create auto-responsive grids with `auto-fill` and `minmax()`
- Understand when to use Grid vs Flexbox

■ CSS Grid replaced every float, table, and clearfix hack. It's the most powerful layout system ever added to CSS. Master it and you can build any 2D layout with clean, readable code.

SECTION 1

Defining a Grid

```
/* ===== GRID CONTAINER ===== */ .grid { display: grid; /* or inline-grid */ /* Define columns */
grid-template-columns: 200px 1fr 2fr; /* 3 columns: fixed, flexible, 2x flexible */
grid-template-columns: repeat(3, 1fr); /* 3 equal columns – shorthand */ grid-template-columns:
repeat(4, minmax(200px, 1fr)); /* 4 columns, min 200px */ /* Define rows */ grid-template-rows:
64px auto 48px; /* header, content, footer */ grid-template-rows: repeat(3, 200px); /* 3 equal
200px rows */ /* Automatic rows for implicitly created rows */ grid-auto-rows: 200px; /*
auto-created rows are 200px tall */ grid-auto-rows: minmax(100px, auto); /* min 100px, grow to
content */ /* Gap between cells */ gap: 24px; /* same for rows and columns */ row-gap: 16px;
column-gap: 24px; /* Align all items within their cells */ justify-items: start | end | center |
stretch; /* horizontal in cell */ align-items: start | end | center | stretch; /* vertical in cell
*/ /* Align the whole grid within the container */ justify-content: start | center | space-between;
align-content: start | center | space-between; }
```

SECTION 2

Placing Grid Items

```
/* Grid lines are numbered from 1 (also -1 from end) */ .item { /* Column placement: start / end
line */ grid-column: 1 / 3; /* span from line 1 to line 3 (2 columns) */ grid-column: 2 / -1; /*
from line 2 to last line */ grid-column: span 2; /* span 2 columns from current position */
grid-column: 1 / -1; /* span ALL columns */ /* Row placement: start / end line */ grid-row: 1 / 3;
/* span 2 rows */ grid-row: span 2; /* span 2 rows from current position */ /* Shorthand: row-start
/ col-start / row-end / col-end */ grid-area: 1 / 1 / 3 / 3; /* Align this item within its grid area
*/ justify-self: start | end | center | stretch; align-self: start | end | center | stretch;
place-self: center; /* both axes at once */ } /* Example: Full-bleed hero */ .hero { grid-column: 1
/ -1; } /* spans all columns */ .sidebar { grid-row: 2 / 4; } /* spans rows 2 and 3 */
```

SECTION 3

Named Grid Areas — Visual Layout Code

```
/* This is the most readable layout code in CSS */ .page { display: grid; grid-template-areas:
"header header header" "sidebar content content" "sidebar content content" "footer footer footer";
grid-template-columns: 250px 1fr 1fr; grid-template-rows: 64px 1fr 1fr 48px; min-height: 100vh;
gap: 0; } /* Assign each element to its named area */ .site-header { grid-area: header; }
.site-sidebar { grid-area: sidebar; } .site-content { grid-area: content; } .site-footer {
grid-area: footer; } /* Dashboard layout */ .dashboard { display: grid; grid-template-areas: "kpi1
kpi2 kpi3 kpi4" "chart chart chart stats" "table table table stats" "table table table alerts";
grid-template-columns: repeat(3, 1fr) 300px; gap: 20px; } .kpi-1 { grid-area: kpi1; } .chart {
grid-area: chart; } /* Dot (.) creates an empty cell */ .minimal { grid-template-areas: "header
header" ". main "; /* left column empty in second row */ }
```

SECTION 4

Auto-Responsive Grid with minmax()

```
/* ===== RESPONSIVE GRID WITHOUT MEDIA QUERIES ===== */ /* auto-fill: create as many columns as
fit, can leave empty tracks */ /* auto-fit: collapse empty tracks – items stretch to fill container
*/ .card-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap:
24px; /* Creates 1, 2, 3, 4+ columns automatically based on screen width */ /* Each column is at
least 280px, max equally-divided fraction of container */ } /* Fixed columns at specific
breakpoints (combine with media queries) */ .product-grid { display: grid; grid-template-columns:
1fr; /* mobile: 1 column */ gap: 16px; } @media (min-width: 600px) { .product-grid {
grid-template-columns: repeat(2, 1fr); } } @media (min-width: 900px) { .product-grid {
```

```
grid-template-columns: repeat(3, 1fr); } } @media (min-width: 1200px) { .product-grid {
grid-template-columns: repeat(4, 1fr); } } /* Masonry-like layout with dense packing */ .mosaic {
display: grid; grid-template-columns: repeat(3, 1fr); grid-auto-rows: 100px; gap: 8px;
grid-auto-flow: dense; /* fill gaps left by spanning items */ } .mosaic__featured { grid-column:
span 2; grid-row: span 2; }
```

SECTION 5

Grid vs Flexbox — When to Use Each

Use Grid When	Use Flexbox When
You need two-dimensional layout (rows AND columns)	You have one-dimensional layout (row OR column)
You want items to align to a defined grid structure	Items should wrap naturally and distribute dynamically
Building page-level layouts (header, sidebar, content)	Building component-level layouts (nav, card internals)
You want named template areas for visual code	You need flexible grow/shrink behaviour
Items need to span multiple columns or rows	Order of items might change (order property)

■ PRACTICE Q & A

Q1. What is the fr unit in CSS Grid?

A: *fr = fractional unit. It represents a fraction of the available space in the grid container AFTER fixed and auto sizes are calculated. grid-template-columns: 1fr 2fr makes the second column twice as wide as the first.*

Q2. What is the difference between auto-fill and auto-fit in repeat()?

A: *auto-fill: creates as many columns as fit, leaving empty tracks. auto-fit: collapses empty tracks — existing items stretch to fill the full width. With enough items, they behave identically.*

Q3. How do you make a CSS Grid item span all columns?

A: *grid-column: 1 / -1. The -1 refers to the last grid line. This works regardless of how many columns the grid has — perfect for full-width hero sections or banners.*

Q4. What does grid-template-areas with a dot (.) do?

A: *A dot creates an EMPTY named cell in the grid. You can use multiple dots for larger empty areas. The named areas still define the grid structure, but no element is assigned to those cells.*

Q5. When should you use CSS Grid over Flexbox?

A: *Use Grid for two-dimensional layouts where items need to align in both rows AND columns. Use Flexbox for one-dimensional content (a row of buttons, a card's internal layout). Modern CSS often uses both: Grid for page layout, Flexbox for components.*

■ CSS Grid Cheat Sheet	
display: grid	Activate CSS Grid
grid-template-columns: repeat(3,1fr)	3 equal columns
grid-template-columns: 200px 1fr	Fixed + flexible column
auto-fill, minmax(280px,1fr)	Auto-responsive columns
gap: 20px	Space between cells
grid-column: 1 / -1	Span all columns
grid-column: span 2	Span 2 columns

grid-row: span 2	Span 2 rows
grid-template-areas	Named visual layout areas
grid-area: name	Assign element to named area
place-items: center	Centre all items in cells
grid-auto-flow: dense	Fill gaps in sparse grids